DEVOPS TRAINING PROGRAM

Beginner · Advanced · Job-Ready

5-6 Months · 4 hrs/week class · 2 hrs/week self-study

Section	Content
Beginner Track	11 weeks · DevOps foundations through IaC
Advanced Track	8 weeks learning + 4 weeks job-seeking
Weekly Pace	4 hrs in-class instruction + 2 hrs self-study including assignment
Monthly Projects	Capstone project at end of each month
Final Project	Major end-to-end production system (Advanced)

BEGINNER

Week	Topic
B1	DevOps, SDLC & How Engineering Teams Work
B2	Git & GitHub: Version Control Fundamentals
B3	Virtualisation: VMs, Containers & Why They Exist
B4	Linux Fundamentals & Essential Commands
B5	AWS Core Services, Principles & Architecture
B6	Shell Scripting with Bash
B7	Python Scripting for DevOps
B8	Docker: Images, Containers & Compose
B9	Git: Branching Strategies & Team Collaboration
B10	CI/CD Concepts & GitHub Actions
B11	Infrastructure as Code: CloudFormation & Terraform

ADVANCED

Week	Topic
A1	AI-Assisted Full-Stack Deploy & System Design
A2	Advanced Linux: Storage & Filesystems
A3	Advanced Linux: Networking & Traffic Flow
A4	Kubernetes: Architecture, Workloads & Operations
A5	Observability: Metrics, Logs, Alerts & Tracing
A6	Advanced CI/CD: Multi-Environment Pipelines & GitOps
A7	AI in DevOps: Tools, Automation & Responsible Use
A8	Major End-to-End Final Project
J1	CV, LinkedIn Finalisation & Portfolio Review
J2	Job Research, Applications & Outreach
J3	Interview Preparation & Mock Interviews
J4	Continuation, Certifications & Community

- Certifications (Optional but Valuable)
  –
  – AWS Certified Solutions Architect Associate (SAA-C03): the gold standard entry-level AWS cert — highly recognised, worth pursuing

CKA (Certified Kubernetes Administrator): the most respected Kubernetes cert — harder, more time investment, but strong signal for SRE and DevOps roles

BEGINNER TRACK

Goal: Go from zero to understanding the core DevOps toolchain — version control, cloud fundamentals, scripting, containers, CI/CD, and infrastructure as code. By the end you will have deployed a real application on AWS with a working pipeline.

BEFORE YOU BEGIN (Optional — Beginner)

This section is entirely optional and designed to help you get the most out of the course. Do as much or as little as you like before Day 1.

LinkedIn (Recommended)

- Create a LinkedIn account if you don't already have one
- Set your headline to something simple like 'Learning DevOps | Cloud Enthusiast'
- Search for and follow 5–10 DevOps engineers — look at how they post, what topics they discuss, what tools they mention. You are not expected to post yet, just observe
- Suggested follows to start: Kelsey Hightower, Liz Fong-Jones, Corey Quinn, Forrest Brazeal

Blogs & Communities to Follow

- dev.to — search 'DevOps beginner' for community articles written by real practitioners
- Medium — search 'AWS for beginners', 'Docker explained', 'what is CI/CD'
- The Cloudcast podcast — good for understanding industry context while commuting
- r/devops on Reddit — read questions others ask; you will recognise many of them later in this course

📌 There is no expectation to write, post, or build a portfolio at the beginner stage. Focus entirely on learning.

WEEK 1 — DevOps, SDLC & How Engineering Teams Work

⏱ **Class:** 4 hours **Self-study (incl. assignment):** 2 hours (including assignment)

Learning Objectives

- Understand what DevOps is and why it exists
- Be able to describe the software development lifecycle (SDLC)
- Know the difference between DevOps, SRE, Platform Engineer, and Cloud Engineer roles
- Understand how a real engineering team operates day-to-day

Core Topics

- **What is DevOps?**
  - The problem DevOps solves: siloed Dev and Ops teams, slow releases, blame culture
  - The DevOps lifecycle: Plan → Code → Build → Test → Release → Deploy → Operate → Monitor
  - DevOps culture: blameless post-mortems, shared ownership, fast feedback loops
- **The SDLC**
  - Waterfall (plan everything upfront) vs Agile (iterate in short cycles)
  - Agile in practice: sprints, standups, retrospectives, backlog
  - Why most companies use Agile today
- **Roles & Career Paths**
  - DevOps Engineer: pipelines, automation, deployment, infrastructure
  - SRE (Site Reliability Engineer): uptime, incident response, reliability metrics
  - Platform Engineer: internal tools and self-service infrastructure for developers
  - Cloud Engineer: cloud architecture, cost, managed services
- **How Engineering Teams Work**
  - Sprint cycles, tickets, code reviews, pull requests, deployments

  - On-call rotations and incident response basics
  - Tools you will encounter: Jira, Confluence, Slack, GitHub

By End of Week — Students Should Have:

- Created an AWS Free Tier account with a billing alert set at $30
- Downloaded Vscode

📺 **Recommended Learning Videos (Self-Study)**
▸ TechWorld with Nana — 'DevOps explained in 8 minutes' (YouTube)
▸ Fireship — 'DevOps CI/CD Explained in 100 seconds' (YouTube)
▸ AWS — 'What is Cloud Computing?' (aws.amazon.com/what-is-cloud-computing)
▸ Recommended read: 'The Phoenix Project' (book) — optional but highly recommended for context

📝 **WEEKLY ASSIGNMENT**
Write a short paragraph (5–8 sentences) in a text file called week1-reflection.txt answering:
• What is DevOps in your own words?
• Which role (DevOps, SRE, Platform, Cloud) interests you most and why?
• What surprised you most about how engineering teams work?

WEEK 2 — Git & GitHub: Version Control Fundamentals

⏱ **Class:** 4 hours **Self-study (incl. assignment):** 2 hours (including assignment)

Learning Objectives

- Understand why version control exists and what problem Git solves
- Use the core Git commands to track changes and push code to GitHub
- Create and manage a GitHub repository
- Basic use of VSCode for github related actions
- Write meaningful commit messages

Core Topics

- **Why Version Control?**
  - The problem: multiple people editing the same file, losing changes, no history
  - Git as a distributed version control system — every developer has the full history
  - Git vs GitHub: Git is the tool, GitHub is the hosting platform
- **Core Git Concepts**
  - Repository (repo): the folder Git tracks
  - The three areas: working directory (your edits), staging area (what you're preparing to commit), commit history (permanent record)
  - Commits: a snapshot of your changes with a message explaining what and why
- **Essential Commands**
  - git init — start tracking a folder
  - git clone <url> — copy a remote repo to your machine
  - git add <file> / git add . — stage changes
  - git commit -m 'message' — save a snapshot
  - git push — upload commits to GitHub
  - git pull — download latest changes from GitHub
  - git status — see what has changed
  - git log — view commit history

- **GitHub Essentials**
  - Creating a repository: public vs private, adding a README and .gitignore
  - The README.md file: what it is, why every repo needs one, basic Markdown syntax
  - .gitignore: telling Git to ignore files (passwords, large files, build outputs)
  - Pushing your first commit: connecting local Git to remote GitHub with SSH or HTTPS
- **Good Commit Messages**
  - Be specific: 'add login form' not 'update file'
  - Present tense: 'fix bug' not 'fixed bug'
  - Conventional commits intro: feat:, fix:, docs: prefixes (you will use these more in Week 9)

By End of Week — Students Should Have:

- A GitHub account repository called 'devops-learning' with their week1-reflection.txt pushed to it
- Made at least 1 commits to understand the add → commit → push cycle

📺 **Recommended Learning Videos (Self-Study)**
▸ The Odin Project — 'Introduction to Git' (theodinproject.com) — read + exercises
▸ Fireship — 'Git explained in 100 seconds' (YouTube)
▸ TechWorld with Nana — 'Git Tutorial for Beginners' (YouTube — first 25 min)
▸ GitHub Skills — 'Introduction to GitHub' (skills.github.com) — free interactive course

📝 **WEEKLY ASSIGNMENT**
Create a public GitHub repo called 'devops-learning'.
Push your week1-reflection.txt from Week 1.
Create a README.md that explains: your name, why you are learning DevOps, and what you hope to build.
Make at least 1 separate commits — each should change something and have a meaningful message.
Share the GitHub repo link in the class chat.

WEEK 3 — Virtualisation: VMs, Containers & Why They Exist

⏱ **Class:** 4 hours **Self-study (incl. assignment):** 2 hours (including assignment)

Learning Objectives

- Explain what a virtual machine is and how hypervisors work
- Understand the difference between VMs and containers
- Explain why containers became the standard for modern application deployment

Core Topics

- **Traditional Servers & The Problem**
  - Physical servers: dedicated hardware, wasteful resource usage, slow to provision
  - The 'works on my machine' problem: different environments cause different behaviour
- **Virtual Machines**
  - What a hypervisor does: software that runs and manages multiple virtual machines on one physical host
  - Type 1 hypervisors (bare-metal): VMware ESXi, Microsoft Hyper-V — run directly on hardware, used in data centres
  - Type 2 hypervisors (hosted): VirtualBox, VMware Workstation — run on top of an OS, used on laptops
  - VMs include a full OS: kernel, system libraries, application — this makes them large and slow to start
  - Use cases for VMs: complete isolation, different OSes, legacy applications, cloud instances (EC2 is a VM)
- **Containers**
  - Containers share the host OS kernel — no need for a full OS per application

  - The Linux features that make containers possible: namespaces (isolation) and cgroups (resource limits)
  - Namespaces: each container gets its own view of processes, network, filesystem, hostname
  - cgroups: limit how much CPU, memory, and I/O a container can use
  - Result: containers start in milliseconds, use far less memory, and are portable
- **VMs vs Containers — Side by Side**
  - VMs: full OS, slow to start (minutes), heavy (GBs), strong isolation
  - Containers: shared kernel, fast to start (seconds), lightweight (MBs), process-level isolation
  - They are not competing: many companies run containers inside VMs for both security and portability
- **The OCI Standard**
  - OCI (Open Container Initiative): the open standard that defines what a container image is and how to run it
  - Why it matters: a container built on your laptop runs identically on AWS, Google Cloud, or your colleague's machine

By End of Week — Students Should Have:

- Be able to explain in plain language, without looking at notes, why containers replaced VMs for application deployment

📺 **Recommended Learning Videos (Self-Study)**
▸ IBM Technology — 'Containers vs VMs: What's the difference?' (YouTube)
▸ TechWorld with Nana — 'Docker vs Virtual Machine' (YouTube)
▸ Fireship — 'Docker in 100 seconds' (YouTube — preview of what's coming)

📝 **WEEKLY ASSIGNMENT**
Add a file called week3-vms-vs-containers.md to your 'devops-learning' GitHub repo.
In your own words (no copy-paste), write:
• What is a hypervisor? (2–3 sentences)
• What Linux features make containers possible? (2–3 sentences)
• Give one real-world scenario where you would choose a VM and one where you would choose a container.
Commit and push. Your explanation should be clear enough for someone who has never heard these terms.

WEEK 4 — Linux Fundamentals & Essential Commands

⏱ **Class:** 4 hours **Self-study (incl. assignment):** 2 hours (including assignment)

Learning Objectives

- Navigate the Linux file system confidently from the command line
- Understand and manage file permissions and ownership
- Connect to a remote server via SSH
- Edit a file using Vim

📌 Linux is the operating system that runs the vast majority of servers, containers, and cloud infrastructure. You don't need to become a Linux expert at this stage — just comfortable enough to work in it.

Core Topics

- **The Linux Shell**
  - The terminal and the shell: your interface to the operating system
  - Bash: the most common shell — what you will use throughout this course
  - The prompt: what user@hostname:directory$ means
  - Important directories: / (root), /home (user files), /etc (config files), /var/log (logs), /tmp (temporary files)

- **Essential Navigation & File Commands**
  - pwd — print working directory (where am I?)
  - ls, ls -la — list files; -l for details, -a for hidden files
  - cd, cd .., cd ~ — change directory, go up, go home
  - mkdir, touch — create directories and files
  - cp, mv, rm — copy, move/rename, delete
  - cat, less — view file contents
  - grep 'pattern' file — search for text in a file
  - man <command> — read the manual for any command
- ***File Permissions***
  - *Three permission types: read (r=4), write (w=2), execute (x=1)*
  - *Three groups: owner, group, everyone else*
  - *Reading ls -la output: -rwxr-xr-- explained*
  - *chmod 755 file — numeric permission setting*
  - *sudo — run commands as administrator*
- **SSH (Secure Shell)**
  - How SSH works: public/private key pair — you keep the private key, the server holds the public key
  - Generating a key pair: ssh-keygen
  - *Connecting: ssh -i key.pem ubuntu@<ip-address> revisited after AWS*
  - *Why SSH keys are more secure than passwords*
- **Vim Basics**
  - Why Vim: it is on every Linux server — you will eventually be forced to use it
  - Modal editing: Normal mode (navigate) vs Insert mode (type)
  - Core commands: i (enter insert), Esc (back to normal), :w (save), :q (quit), :wq (save and quit), :q! (quit without saving)
  - That is all you need to know for now — Vim mastery comes with time

By End of Week — Students Should Have:

- Launched an EC2 t2.micro (Free Tier), connected via SSH, and navigated the file system
- Created, edited (with Vim), and deleted files via the command line

📺 Recommended Learning Videos (Self-Study)
▸ NetworkChuck — 'Linux for Hackers (the Beginner course)' — Ep 1 & 2 (YouTube)
▸ TechWorld with Nana — 'Linux Command Line Tutorial' (YouTube — first 30 min)
▸ OpenVim.com — interactive browser-based Vim tutorial (free, takes ~15 min)

📝 WEEKLY ASSIGNMENT
Launch a free-tier EC2 instance (Amazon Linux 2), connect via SSH.
Complete the following and screenshot each step:
1. Navigate to /var/log and use cat or less to view a log file
2. Create a directory ~/devops-lab/ with two files: notes.txt and commands.sh
3. Set notes.txt to 644 permissions and commands.sh to 755 — verify with ls -la
4. Open notes.txt in Vim, write 3 Linux commands you learned, save and exit
Push a markdown file 'week4-linux-notes.md' to your GitHub repo documenting the commands you used and what they do.

WEEK 5 — AWS Core Services, Principles & Architecture Fundamentals

⏱ **Class:** 4 hours **Self-study (incl. assignment):** 2 hours (including assignment)

Learning Objectives

- Navigate AWS and understand its global infrastructure model
- Use EC2, S3, and IAM confidently for common tasks
- Understand the AWS Shared Responsibility Model
- Know the foundational principles of scalability, reliability, and security on AWS
- Setup AWS CLI and SSH

Core Topics

- **AWS Global Infrastructure**
  - Regions: geographical areas with multiple data centres (e.g. eu-west-1 = Ireland)
  - Availability Zones (AZs): physically separate data centres within a region — deploying across AZs means a fire or power cut in one doesn't take down your service
  - Edge Locations: CloudFront CDN points of presence for serving content close to users
- **EC2 (Elastic Compute Cloud)**
  - EC2 instances are virtual machines — you choose the OS, CPU, RAM
  - Instance types: t3.micro (free tier, general purpose), t3.small, t3.medium
  - Key concepts: AMI (the OS image), security groups (firewall rules), key pairs (SSH access)
  - Launching an instance: choosing AMI → instance type → network → security group → key pair
  - Elastic IP: a static IP that stays even if you stop/start the instance
- **S3 (Simple Storage Service)**
  - Object storage: store any file (images, backups, logs, static websites) as an 'object' with a key
  - Buckets: containers for objects — bucket names are globally unique
  - Key operations: upload, download, delete, list objects
  - Use cases: static website hosting, CI/CD artefact storage, backup, log archives
  - Block public access: enabled by default — explicitly required to make objects public
- **IAM (Identity & Access Management)**
  - IAM controls who can do what in your AWS account
  - Root account: the all-powerful account used only for billing — never for day-to-day work
  - IAM users: accounts for people; IAM roles: accounts for services (EC2, Lambda)
  - Policies: JSON documents that say Allow or Deny on specific actions and resources
  - Least privilege principle: only grant the minimum permissions needed — a key rule in AWS security
  - MFA (Multi-Factor Authentication): always enable on your root and admin accounts
- **AWS Shared Responsibility Model**
  - AWS is responsible for: the physical hardware, data centres, the hypervisor, managed service infrastructure
  - You are responsible for: your OS configuration, your application, your data, IAM permissions, security groups
  - Rule of thumb: AWS secures the cloud, you secure what is IN the cloud
- **Scalability, Reliability & Security — The Fundamentals**
  - Scalability: your system's ability to handle more load. Vertical (bigger server) vs horizontal (more servers)
  - Reliability: avoiding downtime. Multi-AZ = if one data centre fails, others keep serving traffic
  - Security: IAM least privilege, encrypted data, locked-down security groups, no public S3 buckets
  - Cost awareness: Free Tier limits — always set a billing alert, always stop resources when not in use
- **The Well-Architected Framework (Introduction)**
  - AWS's 6-pillar framework for building good systems: Operational Excellence, Security, Reliability, Performance, Cost, Sustainability
  - You don't need to memorise this now — you will revisit it throughout the course

By End of Week — Students Should Have:

- Launched an EC2 instance with a correctly scoped security group (only port 22 from your IP)
- Created an S3 bucket, uploaded a file, and understood why it is not publicly accessible by default
- Created an IAM user with limited permissions and logged in as that user

📺 Recommended Learning Videos (Self-Study)

- ▸ TechWorld with Nana — 'AWS Tutorial for Beginners' (YouTube — first 45 min)
- ▸ AWS — 'AWS Shared Responsibility Model' (docs.aws.amazon.com) — read the official page

▸ Fireship — 'AWS S3 in 100 seconds' (YouTube)
▸ freeCodeCamp — 'AWS Certified Cloud Practitioner' full course (YouTube — use as reference, not required to finish)

📝 **WEEKLY ASSIGNMENT**
Task 1 — EC2: Launch a t2.micro EC2 running Amazon Linux. Configure the security group to allow SSH from your IP only (not 0.0.0.0/0). Connect via SSH and verify you're on the instance.
Task 2 — S3: Create an S3 bucket. Upload a text file. Try to access it publicly — it should fail. Create a bucket policy that allows public read on that specific object. Verify it works.
Task 3 — IAM: Create an IAM user with a policy that allows only s3:GetObject on your specific bucket. Test that this user cannot create EC2 instances or access other services.
Document what you did in a file 'week5-aws-notes.md' and push to GitHub.

WEEK 6 — Shell Scripting with Bash

⏱ **Class:** 4 hours **Self-study (incl. assignment):** 2 hours (including assignment)

Learning Objectives

- Write Bash scripts to automate repetitive tasks
- Use variables, conditionals, loops, and functions in Bash
- Write scripts that check system state and take action based on what they find

📌 4 hours is enough to learn practical Bash scripting for DevOps. Focus on scripts you would actually use: health checks, deploy helpers, log parsers.

Core Topics

- **Bash Script Basics**
  - The shebang: #!/bin/bash — tells the OS to use Bash to run this file
  - Making a script executable: chmod +x script.sh
  - Running a script: ./script.sh or bash script.sh
  - Comments: # this is a comment
  - echo: printing output to the terminal
- **Variables**
  - Defining: NAME='Alice' (no spaces around =)
  - Using: echo $NAME or echo ${NAME}
  - Special variables: $0 (script name), $1 $2 (arguments), $? (exit code of last command), $$ (process ID)
  - Environment variables: existing variables like $HOME, $PATH, $USER
- ***Conditionals***
  - *if [condition]; then ... fi*
  - *File tests: -f (file exists), -d (directory exists), -z (string is empty)*
  - *Comparisons: -eq, -ne, -gt, -lt for numbers; == and != for strings*
  - *Exit codes: 0 means success, non-zero means failure — this is how scripts signal errors*
- ***Loops***
  - *for loop: for item in list; do ... done*
  - *while loop: while [condition]; do ... done*
  - *Looping over files: for file in /logs/*.log; do echo $file; done*
- ***Functions***
  - *my_function() { ... } — defining a function*
  - *Calling: my_function or my_function arg1 arg2*
  - *Why functions: avoid repeating code, make scripts readable*
- **Practical DevOps Patterns**
  - Checking if a service is running and restarting if not

  - Checking disk usage and sending a warning if above a threshold
  - Reading a list of servers from a file and running a command on each
  - Logging script output with timestamps: echo "$(date) - message" >> script.log

By End of Week — Students Should Have:

- Written and run at least ONE Bash scripts on their EC2 instance

📺 **Recommended Learning Videos (Self-Study)**
▸ Fireship — 'Bash in 100 seconds' (YouTube)
▸ TechWorld with Nana — 'Shell Scripting Crash Course' (YouTube — first 30 min)
▸ devhints.io/bash — Bash cheat sheet — bookmark this, refer to it constantly

📝 **WEEKLY ASSIGNMENT**
Write a script called health_check.sh that:
1. Checks disk usage — if above 80%, prints 'WARNING: Disk at X%'
2. Checks if the nginx service is running — if not, prints 'ERROR: nginx is not running'
3. Prints 'All checks passed' if everything is fine
4. Logs every run with a timestamp to a file: /var/log/health_check.log
Run it on your EC2 instance, confirm it works, and push it to your GitHub repo with a README explaining how to use it.

WEEK 7 — Python Scripting for DevOps

⏱ **Class:** 4 hours **Self-study (incl. assignment):** 2 hours (including assignment)

Learning Objectives

- Write basic Python scripts to automate cloud tasks
- Use the boto3 library to interact with AWS programmatically
- Understand when to use Python vs Bash

📌 You are not becoming a Python developer. The goal is to be able to write automation scripts and use AWS SDKs. Focus on being practical, not perfect.

Core Topics

- **Python for DevOps — Why Python?**
  - First-class support for AWS, Kubernetes, Ansible, and most DevOps tooling
  - More readable than Bash for complex logic — easier to maintain
  - Rule of thumb: use Bash for simple one-liners and small scripts; use Python when you need AWS SDK, JSON handling, or complex logic
- **Python Basics (condensed)**
  - Variables, strings, numbers, booleans — Python is dynamically typed
  - Lists: my_list = ['a', 'b', 'c'] — ordered, mutable
  - Dictionaries: my_dict = {'key': 'value'} — key-value pairs, like JSON
  - for loops, if/elif/else, functions with def
  - f-strings: f'Hello {name}' — the clean way to build strings
  - import: loading libraries — import json, import os, import sys
- **Working with Files & JSON**
  - Reading a file: open('file.txt', 'r').read()
  - Parsing JSON: json.loads() and json.dumps()
  - Why JSON matters in DevOps: AWS API responses, config files, and pipeline artefacts are all JSON

- **boto3 — AWS SDK for Python**
  - What boto3 is: the official Python library for interacting with every AWS service
  - import boto3 → create a client or resource for a service
  - Listing EC2 instances: boto3.client('ec2').describe_instances()
  - Listing S3 buckets: boto3.client('s3').list_buckets()
  - Authentication: boto3 uses your AWS CLI credentials or IAM role automatically — no hardcoded keys
- **Virtual Environments & Dependencies**
  - python -m venv venv — create an isolated Python environment
  - source venv/bin/activate — activate it
  - pip install boto3 — install packages without affecting the system Python
  - requirements.txt: list your dependencies so others can reproduce your environment

By End of Week — Students Should Have:

- A Python script that queries AWS and returns useful information

📺 Recommended Learning Videos (Self-Study)
▸ TechWorld with Nana — 'Python for Beginners — Learn Python from Scratch' (YouTube — first 30 min)
▸ Corey Schafer — 'boto3 Python Tutorial' (YouTube)
▸ Real Python — 'Python boto3 AWS guide' (realpython.com) — excellent reference

📝 WEEKLY ASSIGNMENT
Write a Python script called aws_inventory.py that:
1. Lists all EC2 instances in your region with: instance ID, instance type, state (running/stopped), and public IP
2. Lists all S3 buckets with their name and creation date
3. Prints everything in a readable format
• Use a virtual environment, include a requirements.txt
• Do NOT hardcode your AWS credentials — use the CLI config or an IAM role
Push to GitHub with a README. Bonus: add a --region flag so users can query any region.

WEEK 8 — Docker: Images, Containers & Compose

⏱ **Class:** 4 hours **Self-study (incl. assignment):** 2 hours (including assignment)

Learning Objectives

- Understand the Docker architecture and how images become containers
- Write a Dockerfile to containerise an application
- Use Docker Compose to run multi-container applications locally

Core Topics

- **Docker Architecture**
  - Docker Engine: the daemon (dockerd) that manages containers, and the CLI client you use
  - Images: read-only templates built in layers — like a blueprint
  - Containers: a running instance of an image — you can run many containers from one image
  - Docker Hub: a public registry of pre-built images (nginx, postgres, node, python)
- **Core Docker Commands**
  - docker pull nginx — download an image
  - docker run -d -p 8080:80 nginx — run a container in background, map port 8080 on host to 80 in container
  - docker ps — list running containers

  - docker exec -it <id> /bin/bash — open a shell inside a running container
  - docker logs <id> — view container output
  - docker stop <id> / docker rm <id> — stop and remove a container
  - docker images / docker rmi <image> — list and remove images
- **Writing Dockerfiles**
  - FROM: the base image (FROM python:3.11-slim)
  - WORKDIR: set the working directory inside the container
  - COPY: copy files from your machine into the image
  - RUN: execute a command during the build (install packages)
  - EXPOSE: document which port the container listens on (informational)
  - CMD: the command to run when the container starts
  - Layer caching: Docker caches each instruction — put things that change least first (COPY package.json before COPY .)
  - .dockerignore: like .gitignore — exclude node_modules, .git, secrets from the build context
- ***Docker Compose***
  - *What it solves: running multiple containers that need to talk to each other (app + database + reverse proxy)*
  - *compose.yaml: defines services, their images, ports, environment variables, and volumes*
  - *docker compose up -d — start all services in background*
  - *docker compose down — stop and remove containers*
  - *docker compose logs -f — follow logs from all services*
  - *Networks in Compose: services on the same Compose project can reach each other by service name*

By End of Week — Students Should Have:

- A containerised application (any language) running locally via docker compose up

📺 Recommended Learning Videos (Self-Study)
▸ TechWorld with Nana — 'Docker Tutorial for Beginners' (YouTube — full video, highly recommended)
▸ Fireship — 'Docker Compose in 12 minutes' (YouTube)
▸ play-with-docker.com — free browser-based Docker playground (no installation needed)

📝 WEEKLY ASSIGNMENT
Write a Dockerfile for a simple web app (use any of: a Python Flask app, a Node.js Express app, or even a static nginx site).
Build the image and run it locally — verify it works in your browser.
Create a docker-compose.yaml with: (1) your web app service, (2) a PostgreSQL database service.
Verify the two containers can communicate (app can reach the database by service name).
Push your Dockerfile, docker-compose.yaml, and app code to a new GitHub repo called 'docker-lab'. Include a README with instructions to run it.

WEEK 9 — Git: Branching Strategies & Team Collaboration

⏱ **Class:** 4 hours **Self-study (incl. assignment):** 2 hours (including assignment)

Learning Objectives

- Use branches to work on features without affecting the main codebase
- Understand and apply common branching strategies used in real engineering teams
- Open and review pull requests, and resolve merge conflicts

Core Topics

- **Branches**

  - What a branch is: a pointer to a specific commit — a separate line of development
  - git branch feature/login — create a branch
  - git checkout feature/login or git switch feature/login — switch to it
  - git checkout -b feature/login — create and switch in one command
  - git branch -d feature/login — delete after merging
- **Merging & Rebasing**
  - git merge: combines two branches, preserving history — safe for shared branches
  - git rebase: replays your commits on top of another branch — cleaner history, use with care on shared branches
  - Merge conflicts: when two people edit the same line — Git marks the conflict, you resolve it manually
  - Resolving conflicts: edit the file, remove the conflict markers (<<<<, ====, >>>>), then git add and git commit
- **Branching Strategies**
  - GitHub Flow (recommended for CI/CD): main is always deployable → create short-lived feature branches → PR → merge to main → deploy
  - Git Flow: longer-lived branches (develop, release, hotfix) — better for versioned software with scheduled releases
  - Trunk-Based Development: everyone commits to main frequently with feature flags — the gold standard for high-velocity DevOps teams
  - Which to use: GitHub Flow is the right default for this course and most modern teams
- **Pull Requests & Code Review**
  - Opening a PR: title, description (what changed and why), screenshots if relevant
  - Reviewing a PR: commenting on specific lines, requesting changes, approving
  - Branch protection rules: require at least one approved review before merging, require passing CI checks
  - Why PRs matter: they are the primary way teams share knowledge and catch bugs before production
- **Conventional Commits (revisit)**
  - feat: new feature — fix: bug fix — docs: documentation — chore: maintenance — ci: pipeline changes
  - Following this convention enables automated changelog generation and semantic versioning

By End of Week — Students Should Have:

- Opened a pull request using GitHub Flow and had a classmate review it
- Intentionally created and resolved a merge conflict

📺 Recommended Learning Videos (Self-Study)
▸ Atlassian — 'Comparing Git branching strategies' (atlassian.com/git/tutorials) — good written reference
▸ TechWorld with Nana — 'Git Branching and Merging' (YouTube)
▸ The Primeagen — 'Git Merge vs Rebase' (YouTube — entertaining and clear)

📝 WEEKLY ASSIGNMENT
Pair with a classmate on GitHub. Each of you:
1. Fork the other person's 'docker-lab' repo
2. Create a feature branch called 'feature/readme-improvements'
3. Make at least 3 commits improving the README (add setup instructions, a diagram description, usage examples)
4. Open a pull request — write a clear description of what you changed and why
5. Review your classmate's PR: leave at least 2 review comments
6. Resolve any conflicts if they arise
Solo option: create two branches on your own repo, make conflicting edits to the same line in both, then resolve the conflict.

WEEK 10 — CI/CD Concepts & GitHub Actions

⏱ **Class:** 4 hours **Self-study (incl. assignment):** 2 hours (including assignment)

Learning Objectives

- Explain what CI/CD is and why it is fundamental to DevOps
- Write a GitHub Actions workflow that runs tests on every push and pull request
- Build and push a Docker image to a registry from a pipeline

Core Topics

- **What is CI/CD?**
  - Continuous Integration (CI): every developer merges their code to main frequently — automated tests run immediately to catch problems early
  - Continuous Delivery (CD): the code is always in a releasable state — deployment is a decision, not a risk
  - Continuous Deployment: every passing commit is automatically deployed to production — no human trigger needed
  - Why it matters: teams that deploy 10x per day catch bugs faster and ship more value than teams that deploy monthly
- **GitHub Actions Core Concepts**
  - Workflows: YAML files stored in .github/workflows/ — each file defines a pipeline
  - Triggers (on:): push, pull_request, schedule, workflow_dispatch (manual trigger)
  - Jobs: a unit of work that runs on a runner machine — jobs can run in parallel or depend on each other (needs:)
  - Steps: individual commands or marketplace actions within a job
  - Runners: the machines that run your jobs — github-hosted (ubuntu-latest) or self-hosted
  - Actions Marketplace: reusable steps others have built — actions/checkout, actions/setup-node, docker/build-push-action
- **Writing a Workflow**
  - on: push: branches: [main] — trigger on push to main only
  - jobs: build: runs-on: ubuntu-latest — define a job on an Ubuntu runner
  - steps: - uses: actions/checkout@v4 — checkout your code (required for almost every workflow)
  - Environment variables: env: MY_VAR: value — scoped to a step, job, or whole workflow
- **Secrets in GitHub Actions**
  - GitHub Secrets: encrypted values stored in repo settings — access with ${{ secrets.MY_SECRET }}
  - Common secrets: DOCKER_USERNAME, DOCKER_PASSWORD, AWS_ACCESS_KEY_ID
  - Never hardcode secrets in YAML — they will be visible in your repo history forever
- ***Pipeline Status Badges***
  - *Adding a badge to your README: one line of markdown that shows pass/fail status — signals to visitors that your code is tested*

By End of Week — Students Should Have:

- A working GitHub Actions pipeline that builds their Docker image on every push to main

📺 **Recommended Learning Videos (Self-Study)**
▸ TechWorld with Nana — 'GitHub Actions Tutorial' (YouTube)
▸ Fireship — 'GitHub Actions in 100 seconds' (YouTube)
▸ GitHub Docs — 'Understanding GitHub Actions' (docs.github.com) — the official reference

📝 **WEEKLY ASSIGNMENT**
Create a GitHub Actions workflow (.github/workflows/ci.yml) for your 'docker-lab' repo that:
1. Triggers on every push to main and on every pull request
2. Checks out the code
3. Builds your Docker image
4. Pushes the image to Docker Hub (use GitHub Secrets for your credentials)
Add a pipeline status badge to your README.

Create a pull request, observe the pipeline running on the PR, then merge it and observe it running again on main.
Bonus: Add a step that only deploys (pushes to Docker Hub) when pushing to main — not on PRs.

WEEK 11 — Infrastructure as Code: CloudFormation & Terraform

⏱ **Class:** 4 hours **Self-study (incl. assignment):** 2 hours (including assignment)

Learning Objectives

- Understand why Infrastructure as Code (IaC) is a DevOps fundamental
- Write a CloudFormation template to provision AWS resources
- Write basic Terraform configuration to provision the same resources
- Understand the difference between CloudFormation and Terraform

Core Topics

- **Why Infrastructure as Code?**
  - The problem with manual: clicking in the console is slow, error-prone, and impossible to repeat exactly
  - IaC solves this: infrastructure defined in files, version-controlled in Git, repeatable and reviewable
  - Declarative IaC: you describe the desired state, the tool figures out how to achieve it
  - Key benefit: disaster recovery — if your AWS account is compromised, you rebuild the entire environment in minutes
- **AWS CloudFormation**
  - AWS-native IaC: free to use, deeply integrated with every AWS service
  - Templates: YAML or JSON files describing AWS resources (EC2, S3, IAM, VPC)
  - Stacks: a CloudFormation deployment — the collection of resources described in one template
  - Key sections: Parameters (inputs), Resources (required — the actual infrastructure), Outputs (values to export)
  - Simple example: a template that creates an S3 bucket with versioning enabled
  - Update and rollback: CloudFormation tracks changes and can roll back automatically if an update fails
- **Terraform Basics**
  - Cloud-agnostic: works with AWS, GCP, Azure, Kubernetes, and hundreds of other providers
  - HCL (HashiCorp Configuration Language): the language Terraform uses — readable and concise
  - Core workflow: terraform init (download providers) → terraform plan (preview changes) → terraform apply (make changes) → terraform destroy (remove everything)
  - Providers: plugins that communicate with cloud APIs — the AWS provider is the most used
  - Resources: the infrastructure you want — resource 'aws_s3_bucket' 'my_bucket' { ... }
  - Variables and outputs: parametrise your configuration, export values for use elsewhere
  - State file: Terraform tracks what it has created in terraform.tfstate — treat this file carefully
- **CloudFormation vs Terraform**
  - CloudFormation: AWS-only, no extra tooling, free, best for AWS-only shops
  - Terraform: multi-cloud, huge community, more features, better for teams working across providers
  - Both are used in industry — knowing both is a competitive advantage

By End of Week — Students Should Have:

- Provisioned an S3 bucket with both a CloudFormation template and a Terraform configuration
- Understood that the same infrastructure can be described in multiple IaC tools

📺 **Recommended Learning Videos (Self-Study)**
▸ TechWorld with Nana — 'Terraform Tutorial for Beginners' (YouTube — first 35 min)
▸ AWS — 'AWS CloudFormation Getting Started' (docs.aws.amazon.com)
▸ Fireship — 'Terraform in 100 seconds' (YouTube)
▸ HashiCorp Learn — 'Get started with Terraform on AWS' (developer.hashicorp.com) — free interactive tutorial

📝 WEEKLY ASSIGNMENT
CloudFormation: Write a template that creates: an S3 bucket with versioning enabled, and an IAM role that allows an EC2 instance to read from that bucket. Deploy it as a stack.
Terraform: Write the equivalent configuration. Provision it with terraform apply. Then run terraform destroy to remove it.
Push both to a folder called 'iac-lab' in your GitHub repo — one subfolder for cloudformation/, one for terraform/.
In a README, explain: what each tool did, one advantage of CloudFormation, one advantage of Terraform.

🎯 BEGINNER CAPSTONE PROJECT (End of Month 3)
Deploy a real application to AWS using everything you have learned.
Requirements:
• A web application (any language) containerised with Docker
• Infrastructure provisioned with Terraform: EC2, S3, IAM role, Security Group
• CI/CD pipeline with GitHub Actions: builds Docker image, pushes to Docker Hub, deploys to EC2
• The application must be accessible via a public URL (EC2 public IP or DNS)
• A clean GitHub repo with: README, architecture description, how-to-run instructions
Deliverable: share the GitHub repo link in the class channel. Present a 5-minute demo.

ADVANCED TRACK

Goal: Operate and scale production-grade systems. Master Kubernetes, observability, security, AI-assisted DevOps, and end-to-end architecture. Graduate job-ready with a polished LinkedIn profile, a strong portfolio, and a CV tailored for DevOps roles.

Phase	Duration & Focus
Month 4	Learning — System Design, Advanced Linux, Kubernetes, Observability
Month 5 (Weeks 1–4)	Learning — Advanced CI/CD, GitOps, AI in DevOps, Final Project
Month 5 (Weeks 5–8)	Job-Seeking — LinkedIn, CV, portfolio review, applications, interview prep

BEFORE YOU BEGIN ⚡ (Mandatory — Advanced)

This section is mandatory for all Advanced track students. These activities are not optional extras — they are the difference between completing a course and being hired. Start them before Week 1 of this track and sustain them throughout.

LinkedIn — Build Your Professional Presence

- Upgrade your headline: '[Current Role or 'Aspiring'] DevOps Engineer | AWS | Kubernetes | CI/CD' — use keywords recruiters search for
- Write your About section: tell your story — why DevOps, what you have built, what you are looking for. First-person, specific, genuine — 3–5 short paragraphs
- Add your skills: AWS, Docker, Kubernetes, Terraform, CI/CD, GitHub Actions, Python, Linux — LinkedIn uses these for recruiter search ranking
- Turn on 'Open to Work' if you are actively looking — it significantly increases recruiter contact
- Post at least once per week throughout this track: what you learned, what you built, a problem you solved. Short posts with a screenshot or diagram perform well
- Comment meaningfully on 5–10 posts per day in your niche — commenting consistently builds your profile visibility faster than posting alone
- Connect with 5 new people per week with a personalised note — class peers, people whose content you read, hiring managers at companies you want to work at

Blog & Writing — Build Authority

- Set up a Medium or dev.to account — publish your first post before the end of Week 2 of this track
- Write one article per month. Topics: a project walkthrough, a tool comparison, a lessons-learned post, a how-I-broke-and-fixed-it story
- Every article should have: a clear problem, your solution, code snippets or diagrams, and honest reflection
- Share every article on LinkedIn with a short personal take — this is the highest-value content for profile visibility
- Don't wait until you feel like an expert. 'I'm learning X and here is what I found confusing and how I figured it out' is more useful to readers than a perfect technical tutorial

Portfolio Building — GitHub as Your CV

- Audit your GitHub: every repo should have a clear README — project purpose, architecture, how to run it, what you learned
- Add an architecture diagram to every significant project — use draw.io, Excalidraw, or Lucidchart (all free)
- Pin your 4–6 best repos to your GitHub profile — a recruiter should understand your skills in 60 seconds
- Everything you build in this track goes on GitHub immediately. A clean commit history that shows real work is evidence of engineering ability

Meetups, Conferences & Community

- Attend at least one in-person or virtual meetup per month: search meetup.com for 'DevOps', 'Kubernetes', 'AWS' in your city
- CNCF (Cloud Native Computing Foundation) virtual events are free and world-class — cncf.io/events
- KubeCon, AWS re:Invent, and HashiConf talks are on YouTube — watch at least one talk per week
- Introduce yourself in communities: CNCF Slack (#kubernetes, #devops), r/devops, the AWS Community

ADVANCED WEEK 1 — AI-Assisted Full-Stack Deploy: System Design for Production

⏱ **Class:** 4 hours **Self-study (incl. assignment):** 2 hours (including assignment)

This week sets the tone for the entire advanced track. Instead of learning tools in isolation, you will use AI to help you deploy a full-stack application to AWS from scratch — then use that experience to understand real system design tradeoffs.

Learning Objectives

- Use AI tools (Copilot, Claude, ChatGPT) effectively to accelerate infrastructure and deployment tasks
- Deploy a full-stack application to AWS (frontend + backend + database)
- Explain system design tradeoffs: scalability vs cost, reliability vs complexity, security vs convenience

Core Topics

- **Using AI as a DevOps Accelerator**
  - What AI is good at: generating boilerplate, explaining error messages, writing Dockerfiles, Terraform, pipeline YAML
  - What AI is bad at: knowing your specific environment, security-sensitive decisions, cutting-edge tool versions — always verify
  - Effective prompting: 'Act as a senior DevOps engineer. I need a Terraform config to provision...' — be specific, include context
  - Using AI to learn, not to skip learning: ask AI to explain what each line of a generated config does
- **Full-Stack Deployment on AWS**
  - Frontend: static site hosted on S3 with CloudFront as CDN
  - Backend: a simple API (Node or Python) running on EC2 behind an ALB
  - Database: RDS (PostgreSQL) in a private subnet — never expose your database to the internet
  - The request flow: user → CloudFront → ALB → EC2 → RDS (you must be able to draw and explain this)
- **System Design: Scalability**
  - Single point of failure: if your one EC2 instance crashes, everything goes down — how do you fix this?
  - Horizontal scaling: Auto Scaling Group puts multiple EC2 instances behind the ALB — traffic is distributed
  - Database scaling: read replicas for read-heavy workloads; vertical scaling for write-heavy (at this scale)
  - Tradeoff: more instances = more cost. This is why right-sizing matters.
- **System Design: Reliability**
  - Multi-AZ: deploy EC2 instances in at least 2 AZs — if one AZ has an outage, the other continues serving traffic
  - RDS Multi-AZ: automatic failover in ~60 seconds if the primary database fails
  - Health checks: ALB continuously checks if EC2 instances are healthy — unhealthy instances are removed automatically
  - Tradeoff: Multi-AZ doubles your compute and database cost. You have to decide if the uptime requirement justifies it.
- **System Design: Security**
  - Network segmentation: frontend is public, backend is in a private subnet (only ALB can reach it), database is in a private subnet (only backend can reach it)
  - IAM roles: EC2 instances use IAM roles to access S3 and Secrets Manager — no hardcoded credentials
  - Secrets Manager: database passwords stored and rotated in Secrets Manager, retrieved at runtime
  - Tradeoff: layered security adds configuration complexity. But a breach is always more expensive than the complexity.
- **System Design: Cost**
  - Identify expensive components: data transfer, NAT Gateway, RDS Multi-AZ, ALB — each has a cost
  - Right-sizing: a t3.small is often enough for a backend serving < 1,000 users/day
  - Free Tier limits: stay aware of what is free and what will charge you
  - Tradeoff: the cheapest architecture is usually the least reliable. Always understand what you are giving up.

By End of Week — Students Should Have:

- A full-stack application running on AWS (S3 + CloudFront + EC2 + RDS)
- An architecture diagram with annotations explaining the scalability, reliability, and security decisions made

📺 Recommended Learning Videos (Self-Study)
▸ AWS — 'AWS Architecture Center' (aws.amazon.com/architecture) — browse reference architectures
▸ TechWorld with Nana — 'AWS Tutorial Full Course' (YouTube — relevant sections on VPC, EC2, RDS)
▸ ByteByteGo — 'System Design: How to design a URL shortener' (YouTube — great intro to system design thinking)
▸ Corey Quinn (Last Week in AWS podcast) — for AWS cost awareness

📝 WEEKLY ASSIGNMENT
Using AI assistance (Copilot, Claude, or ChatGPT): deploy a full-stack application to AWS.
The architecture must include: S3 + CloudFront for frontend, EC2 (or ECS) for backend API, RDS for database, ALB, proper security groups.
Document every AI prompt you used and what it generated — note where the AI was helpful and where you had to correct it.
Draw an architecture diagram. Annotate it with one sentence per component explaining: what it does, why you chose it, and one tradeoff.
Push everything to GitHub. This becomes the base application for your final project.

ADVANCED WEEK 2 — Advanced Linux: Storage & Filesystem

⏱ **Class:** 4 hours **Self-study (incl. assignment):** 2 hours (including assignment)

Learning Objectives

- Understand Linux storage: block devices, filesystems, and mounting
- Manage EBS volumes on AWS from the Linux command line
- Understand the Linux storage stack and how containers use it

Core Topics

- **The Linux Storage Stack**
  - Block devices: the raw storage hardware or virtual device — /dev/xvda, /dev/nvme0n1
  - Partitions: dividing a block device — /dev/xvda1, /dev/xvda2
  - Filesystems: the structure written onto a partition — ext4 (most common), xfs, btrfs
  - Mount points: making a filesystem accessible at a path — mounting /dev/xvda1 at /
- **Key Commands**
  - lsblk — list block devices and their mount points in a tree view
  - df -h — disk space used and available per mounted filesystem
  - du -sh /path — disk usage of a specific directory
  - fdisk -l — list partitions on a device
  - mkfs.ext4 /dev/xvdf — format a device with ext4 filesystem
  - mount /dev/xvdf /data — mount a device at a path
  - umount /data — unmount
  - /etc/fstab — persistent mount configuration (survives reboots)
- **AWS EBS (Elastic Block Store)**
  - EBS volumes are network-attached block storage — they appear as disk devices to Linux
  - Attaching an EBS volume to EC2: in the console or with AWS CLI/boto3
  - After attaching: the device appears but is unformatted — you must mkfs and mount it

  - EBS volume types: gp3 (general purpose SSD, default), io2 (high IOPS), st1 (throughput HDD for logs/big data)
  - Snapshots: point-in-time backup of an EBS volume — stored in S3, used for disaster recovery and AMI creation
- **Storage in the Container Context**
  - Containers have ephemeral storage: data written inside a container is lost when the container stops
  - Docker volumes and bind mounts: persisting data outside the container lifecycle
  - The container storage driver (overlay2): how Docker manages layered images on the host filesystem
  - EBS + Docker: attaching an EBS volume to an EC2 instance and mounting it as a Docker volume path

📺 **Recommended Learning Videos (Self-Study)**
▸ TechWorld with Nana — 'Linux File System explained' (YouTube)
▸ AWS — 'Amazon EBS Getting Started' (docs.aws.amazon.com)
▸ Linux Journey — 'Filesystem' module (linuxjourney.com) — free interactive course

📝 **WEEKLY ASSIGNMENT**
Attach a new EBS volume (5 GiB gp3) to your EC2 instance.
From the command line: confirm the device appears with lsblk, format it with ext4, mount it at /data, and verify with df -h.
Add the mount to /etc/fstab so it persists after a reboot. Reboot and confirm it re-mounts automatically.
Create a Docker container that writes data to a path mapped to /data. Stop and remove the container, verify the data persists.
Take an EBS snapshot. Document the full process in a GitHub markdown file with the commands you used.

*ADVANCED WEEK 3 — Advanced Linux: Networking & Traffic Flow *** may be removed*

⏱ ***Class:*** *4 hours* ***Self-study (incl. assignment):*** *2 hours (including assignment)*

Learning Objectives

- *Trace how a network packet moves from a browser to a running container*
- *Understand the Linux networking stack: interfaces, routing, iptables*
- *Diagnose common network issues using Linux tools*

📌 *The goal is not to become a network engineer. The goal is to be able to answer 'why can't my service reach this other service?' — a question you will face regularly.*

Core Topics

- ***Network Interfaces & IP***
  - *ip addr — list network interfaces and their IP addresses (replaces ifconfig)*
  - *lo: the loopback interface (127.0.0.1) — only accessible from the same machine*
  - *eth0 / ens5: the primary network interface — has the instance's private IP*
  - *ip route — show the routing table: which interface handles traffic to which destination*
- ***Understanding Traffic Flow***
  - *A request enters your EC2 from the internet: internet → Internet Gateway → VPC routing → EC2 eth0 interface → Linux networking stack → your application process*
  - *Port forwarding in Docker: iptables NAT rules are created to forward traffic from host port 8080 to container port 80*
  - *How to read this: docker run -p 8080:80 creates an iptables rule — you can see it with iptables -t nat -L -n*
- ***Diagnostic Tools***
  - *ping <host> — check basic connectivity (ICMP)*
  - *curl -v http://host:port — make HTTP requests and see full request/response headers*
  - *ss -tulpn — show all listening ports and which process owns them (replaces netstat)*
  - *traceroute <host> — show each hop a packet takes to reach a destination*
  - *tcpdump -i eth0 port 80 — capture packets on an interface — the ultimate debugging tool*

  - nslookup / dig — DNS resolution queries
- **DNS**
  - How DNS works: your app asks a resolver → resolver queries root servers → TLD servers → authoritative nameserver → returns IP
  - Common record types: A (hostname → IPv4), CNAME (alias), MX (mail), TXT (verification)
  - DNS TTL: time-to-live — how long resolvers cache the answer. During deployments, low TTL allows fast DNS changes
  - AWS Route 53: DNS service, health checks, failover routing — covered more in the final project context
- **Security Groups vs iptables**
  - Security Groups operate at the AWS network level — before traffic even reaches your EC2 instance
  - iptables operates inside the Linux OS — after traffic has been allowed by the security group
  - Both can block traffic — always check both when debugging connectivity issues

📺 **Recommended Learning Videos (Self-Study)**
▸ Hussein Nasser — 'How the internet works' series (YouTube — clear and deep)
▸ TechWorld with Nana — 'Networking for DevOps Engineers' (YouTube)
▸ Julia Evans — networking zines (jvns.ca) — free, approachable, highly recommended

📝 **WEEKLY ASSIGNMENT**
Run a container on your EC2 on port 8080. Use tcpdump to capture the traffic when you curl it from outside.
Use ss -tulpn to list all listening ports. Identify which process owns each port.
Simulate a DNS lookup failure: add a fake entry to /etc/hosts, curl a hostname that resolves to your fake IP — observe the error. Remove the entry and verify it works again.
Use dig to trace the full DNS resolution chain for any public domain. Document each step: resolver → root → TLD → authoritative nameserver.
Write a one-page networking troubleshooting runbook: 'My container is not reachable — here are the 5 things I check in order.'

ADVANCED WEEK 3-4 — Kubernetes: Architecture, Workloads & Operations

⏱ **Class:** 4 hours **Self-study (incl. assignment):** 2 hours (including assignment)

Learning Objectives

- Explain the Kubernetes architecture and what each component does
- Deploy and manage applications in Kubernetes using core resources
- Scale, update, and debug Kubernetes workloads

Core Topics

- **Why Kubernetes?**
  - Docker Compose is great for local development, but not for production: no self-healing, no auto-scaling, no rolling updates
  - Kubernetes solves: automatic restarts, scaling, rolling deployments, service discovery, configuration management — at scale
- **Kubernetes Architecture**
  - Control Plane: the brain — API Server (receives all commands), etcd (stores all state), Scheduler (assigns pods to nodes), Controller Manager (reconciles actual vs desired state)
  - Data Plane: the workers — Nodes run your containers via the kubelet (node agent), kube-proxy (networking), and container runtime (containerd)
  - kubectl: the CLI — communicates with the API Server to inspect and manage everything
- **Core Resources**

  - Pod: the smallest deployable unit — one or more containers that always run together on the same node
  - Deployment: manages a set of identical Pods — defines desired replicas, rolling update strategy, rollback
  - Service: gives your Pods a stable network identity (DNS name and IP) even as Pods are replaced — types: ClusterIP, NodePort, LoadBalancer
  - ConfigMap: store configuration that is not secret — mounted as env vars or files
  - Secret: store sensitive data (base64 encoded, not encrypted by default — use Sealed Secrets or CSI driver for real security)
  - Namespace: logical partition within a cluster — organise workloads, apply resource quotas
- **Scaling & Updates**
  - kubectl scale deployment/my-app --replicas=5 — scale manually
  - HPA (Horizontal Pod Autoscaler): scale based on CPU or memory metrics automatically
  - Rolling update: Kubernetes replaces old Pods with new ones gradually — zero downtime by default
  - kubectl rollout status / kubectl rollout undo — check rollout progress and revert if needed
- **Debugging in Kubernetes**
  - kubectl describe pod/name — full event log, environment variables, volume mounts, status — start here for any issue
  - kubectl logs pod/name — container stdout. kubectl logs --previous — logs from crashed previous container
  - kubectl exec -it pod/name -- /bin/sh — shell into a running pod
  - Common failure: CrashLoopBackOff — container keeps crashing. kubectl logs --previous to see why.
  - Common failure: ImagePullBackOff — can't pull the Docker image. Wrong tag? Private registry? Wrong credentials?
- **Ingress & EKS**
  - Ingress: route external HTTP/HTTPS traffic to internal Services based on hostname or path
  - Ingress Controller: the component that implements Ingress — nginx ingress controller or AWS Load Balancer Controller
  - EKS (Elastic Kubernetes Service): AWS-managed Kubernetes — AWS manages the control plane, you manage the node groups
  - eksctl: the quickest way to provision an EKS cluster from the command line

📺 Recommended Learning Videos (Self-Study)
▸ TechWorld with Nana — 'Kubernetes Tutorial for Beginners' (YouTube — full video, 4 hours, watch in sections)
▸ Fireship — 'Kubernetes explained in 100 seconds' (YouTube)
▸ KillerCoda — free interactive Kubernetes scenarios in your browser (killercoda.com)
▸ CNCF — 'Kubernetes Basics' interactive tutorial (kubernetes.io/docs/tutorials)

📝 WEEKLY ASSIGNMENT
Using kubeadm for cluster creation. Create a cluster.
Deploy your Week 1 application using: Deployment (3 replicas), Service (ClusterIP), ConfigMap for config, Ingress for HTTP routing.
Scale to 3 replicas. Watch kubectl get pods -w. Scale back to 2.
Deliberately break a pod (wrong image tag) — observe CrashLoopBackOff, use kubectl logs and kubectl describe to diagnose, fix it.
Add a HPA that scales between 2 and 8 replicas based on CPU. Generate load with curl in a loop and watch it scale.
All manifests in a /k8s folder in your GitHub repo with a README.

ADVANCED WEEK 5 — Observability: Metrics, Logs, Alerts & Tracing

⏱ **Class:** 4 hours **Self-study (incl. assignment):** 2 hours (including assignment)

Learning Objectives

- Understand the three pillars of observability and why each matters

- Deploy Prometheus and Grafana to monitor a Kubernetes cluster
- Write alert rules that catch real issues without drowning you in noise

Core Topics

- **Observability Philosophy**
  - Monitoring tells you something is wrong. Observability tells you why.
  - The Three Pillars: Metrics (numbers over time), Logs (events that happened), Traces (how time was spent across services)
  - The four golden signals (Google SRE): Latency, Traffic, Errors, Saturation — start with these
  - SLI / SLO / SLA: SLI is what you measure (e.g. request success rate), SLO is your target (99.9%), SLA is the contractual commitment
- **Metrics with Prometheus**
  - Prometheus scrapes /metrics endpoints on a schedule and stores time-series data
  - Metric types: Counter (only goes up — request count), Gauge (up/down — current memory), Histogram (distributions — latency buckets)
  - PromQL basics: rate(http_requests_total[5m]) — per-second request rate over 5 min window
  - kube-prometheus-stack Helm chart: installs Prometheus + Grafana + Alertmanager + node_exporter in one command
  - Service discovery: Prometheus finds pods to scrape via Kubernetes service discovery — annotate pods with prometheus.io/scrape: 'true'
- **Dashboards with Grafana**
  - Data sources: connect to Prometheus, then build panels using PromQL queries
  - Import community dashboards: grafana.com/grafana/dashboards — Node Exporter Full (ID 1860) is a great starting point
  - Dashboard variables: make dashboards interactive (filter by namespace, pod, node)
- **Alerting**
  - Alertmanager: handles routing, grouping, and deduplication of Prometheus alerts
  - Alert rules: defined in PrometheusRule resources — alert when a PromQL expression is true for a duration
  - Good alerts are actionable: 'CPU > 90% for 5 minutes' means something needs to be done. 'CPU > 50%' is noise.
  - Routing: Slack for warnings, PagerDuty for critical — don't page someone for an alert they can ignore
- **Logging (Overview)**
  - Structured logging: JSON format — machine-readable, enables powerful queries
  - Loki: log aggregation that integrates with Grafana — like Prometheus for logs
  - Promtail: collects logs from pods and sends to Loki
  - CloudWatch Logs: AWS-native — useful for Lambda, RDS, VPC Flow Logs even when using Loki for application logs

📺 Recommended Learning Videos (Self-Study)
▸ TechWorld with Nana — 'Prometheus and Grafana Tutorial' (YouTube — full walkthrough)
▸ Grafana Labs — 'Introduction to Prometheus' (grafana.com/learn)
▸ Charity Majors — 'Observability 101' talks on YouTube — essential viewing for the philosophy

📝 WEEKLY ASSIGNMENT
Deploy kube-prometheus-stack to your kind or EKS cluster using Helm.
Build a Grafana dashboard with: cluster CPU & memory, pod count, HTTP request rate for your app (if it exposes /metrics).
Write an alert rule that fires when any pod is in CrashLoopBackOff state. Route the alert to a Slack webhook or email.
Write one PromQL query that returns: the 5-minute request rate for your application, broken down by HTTP status code.
Screenshot your dashboard and post it on LinkedIn: 'Week X of my DevOps training — just set up production-grade monitoring.'

ADVANCED WEEK 7 — Advanced CI/CD: Multi-Environment Pipelines & GitOps

⏱ **Class:** 4 hours **Self-study (incl. assignment):** 2 hours (including assignment)

Learning Objectives

- Design a multi-environment pipeline with promotion gates
- Implement GitOps with ArgoCD for Kubernetes continuous delivery
- Manage secrets securely across environments and pipelines

Core Topics

- **Multi-Environment Pipelines**
  - Environments: dev (auto-deploy on every merge), staging (deploy with approval), production (deploy with approval + delay)
  - GitHub Environments: define environments in repo settings, add required reviewers and wait timers
  - Environment-specific config: use different .tfvars files and secrets per environment — not different branches
  - Promotion: passing a tested Docker image tag from dev → staging → prod (the same image, not rebuilt)
- **GitOps Principles**
  - Git as the single source of truth: the desired state of your cluster is always a Git commit
  - Declarative: everything in Git describes what should exist — the system automatically makes reality match Git
  - Observable: any drift between Git and the running cluster is visible and automatically corrected
  - Why GitOps matters: full audit trail, easy rollback (git revert), consistent state across environments
- **ArgoCD**
  - ArgoCD watches a Git repo for Kubernetes manifest changes and syncs them to your cluster automatically
  - Application: an ArgoCD resource that says 'watch this repo path and sync it to this cluster/namespace'
  - Sync policies: auto-sync vs manual — start with manual to understand, then enable auto
  - Application health: ArgoCD shows whether pods are running, degraded, or unknown — a single pane of glass
  - Rollback: git revert the bad commit, ArgoCD detects the change and syncs the previous state back
- **Secrets in Pipelines & GitOps**
  - GitHub OIDC: the right way to authenticate GitHub Actions with AWS — assumes an IAM role per run, no long-lived credentials
  - External Secrets Operator: watches Kubernetes and syncs secrets from AWS Secrets Manager into Kubernetes Secrets automatically
  - Sealed Secrets: encrypt Kubernetes Secrets so they can safely be committed to Git
  - gitleaks in CI: scans every commit for accidentally committed secrets — add this to every pipeline

📺 Recommended Learning Videos (Self-Study)
▸ TechWorld with Nana — 'ArgoCD Tutorial for Beginners' (YouTube)
▸ Weaveworks — 'Introduction to GitOps' (weave.works) — the original GitOps explainer
▸ GitHub Docs — 'Using environments for deployment' (docs.github.com) — read the official docs

📝 WEEKLY ASSIGNMENT
Install ArgoCD on your EKS cluster. Create an ArgoCD Application that watches your /k8s directory in GitHub.
Configure it to auto-sync: make a change to a Kubernetes manifest in GitHub, watch ArgoCD detect and apply it without you running kubectl.
Add a multi-environment GitHub Actions pipeline: dev deploys automatically, staging requires one approval, production requires two approvals.
Integrate External Secrets Operator to pull your database password from AWS Secrets Manager into a Kubernetes Secret.

Add gitleaks to your pipeline. Intentionally commit a fake secret (e.g., AWS_KEY=AKIAFAKEKEY123456789), observe gitleaks catch it, remove it.

ADVANCED WEEK 7 — AI in DevOps: Tools, Automation & Responsible Use

⏱ **Class:** 4 hours **Self-study (incl. assignment):** 2 hours (including assignment)

Learning Objectives

- Use AI tools to accelerate real DevOps workflows without losing engineering judgment
- Build an AI-augmented pipeline feature: PR descriptions, runbook generation, or intelligent alerting
- Understand the risks of AI in production systems and how to mitigate them
- MCPs

Core Topics

- **AI Tools for DevOps Engineers (Practical)**
  - GitHub Copilot: autocomplete for Dockerfiles, Terraform, Kubernetes YAML, pipeline configs — dramatically faster boilerplate
  - Claude / ChatGPT: paste error messages and ask for diagnosis; ask 'explain this Kubernetes event'; ask 'write a Terraform module for...'
  - Cursor / Continue: AI-native editors that understand your codebase context, not just the current file
  - Best prompt pattern: 'I am a DevOps engineer. I am trying to [goal]. I have [context]. The error I'm seeing is [error]. What should I check?'
- **AI for Infrastructure & Automation**
  - AI-generated IaC: use AI to generate Terraform or CloudFormation — always review and understand what it produces before applying to production
  - AI-assisted debugging: paste CloudWatch logs or Kubernetes events into an AI and ask it to identify the root cause
  - Auto-remediation: Lambda functions triggered by CloudWatch alarms to automatically fix known, safe issues (e.g., restart a stuck service)
  - AIOps: anomaly detection in metrics and logs — AWS DevOps Guru, Datadog Watchdog — ML models that learn normal behaviour and alert on deviation
- **Building AI-Augmented Pipeline Features**
  - AI-generated PR descriptions: a GitHub Actions step that calls an LLM API with the git diff and posts a summary as a PR comment
  - AI-generated release notes: summarise commits between tags into human-readable release notes automatically
  - AI code review: CodeRabbit, Greptile — AI reviews PRs and leaves comments before the human reviewer sees them
  - Runbook chatbot: a Slack bot that responds to '/runbook nginx-crash' by querying your documentation with RAG (Retrieval-Augmented Generation)
- **AI for Learning (Meta-skill)**
  - Using AI to explain concepts: 'explain eBPF as if I'm a DevOps engineer who knows Linux basics'
  - Using AI to review your architecture: 'here is my architecture diagram — what are the security risks?'
  - Using AI to prepare for interviews: 'give me 5 hard DevOps interview questions about Kubernetes networking'
- **Risks & Responsible Use**
  - Never paste production data, customer data, or real secrets into AI tools — treat them as untrusted third parties
  - AI is confidently wrong: hallucinated AWS API calls, outdated Terraform syntax, wrong Kubernetes versions — always verify against official docs
  - Over-reliance degrades skills: use AI to understand and learn, not to skip understanding — you will be asked to explain the code in an interview
  - AI in critical systems: always require human approval before AI-suggested changes reach production

📺 **Recommended Learning Videos (Self-Study)**

- ▸ GitHub — 'GitHub Copilot for DevOps' (YouTube — official demo)
- ▸ Fireship — 'AI tools every developer should know in 2025' (YouTube)
- ▸ AWS — 'Amazon DevOps Guru' documentation (docs.aws.amazon.com)

▸ Andrej Karpathy — 'Intro to Large Language Models' (YouTube — understanding what AI actually is)

📝 **WEEKLY ASSIGNMENT**
Build one AI-powered feature and add it to your pipeline. Choose one:
Option A: AI PR Descriptions — a GitHub Actions step that calls Claude or OpenAI with the git diff and posts a summary comment on every PR.
Option B: AI Runbook Bot — a Lambda function that responds to a Slack slash command with a relevant runbook, using AWS Bedrock.
Option C: AI Alert Summariser — a Lambda triggered by a CloudWatch alarm that calls an LLM, summarises the alert context, and posts it to Slack with suggested next steps.
Document: what AI tool you used, how you prompted it, what it generated, what you had to fix or verify, and one thing it got wrong.
Write a LinkedIn post: 'How I use AI in my DevOps workflow — what helps and what to watch out for.'

ADVANCED WEEK 8 — Major End-to-End Final Project

⏱ **Class:** 4 hours class (workshop/Q&A format) **Self-study (incl. assignment):** Extended — 2 weeks to complete

This is the capstone of the entire program. You will evolve your Week 1 application into a production-grade system demonstrating mastery of everything covered. This project is your primary portfolio piece and the centrepiece of your job applications.

Project Requirements

- **Infrastructure (Terraform)**
  - Full AWS environment: VPC (2+ AZs), EKS cluster, RDS Multi-AZ, ElastiCache (Redis), ALB, S3, CloudFront
  - All resources in Terraform modules — nothing created by clicking in the console
  - Remote Terraform state in S3 with DynamoDB locking
- **Application & Kubernetes**
  - At least 2 services (e.g. frontend + API) deployed to EKS with separate Deployments
  - Kubernetes: Deployments, Services, Ingress with TLS (ACM certificate), ConfigMaps, Secrets via External Secrets Operator, HPA, Network Policies
  - All container images in ECR, pushed from pipeline (not built on the cluster)
- **CI/CD (GitHub Actions + ArgoCD)**
  - CI: lint → test → Semgrep (SAST) → Trivy (container scan) → tfsec (IaC scan) on every PR
  - CD: build → push ECR → ArgoCD auto-syncs to dev → manual approval to staging → manual approval to prod
  - At least one AI-augmented step in the pipeline
- **Observability**
  - kube-prometheus-stack deployed, Grafana dashboard covering cluster health and application metrics
  - At least 2 configured alert rules routed to Slack or email
  - Loki + Promtail for log aggregation (or CloudWatch for AWS-native logging)
- **Security**
  - IAM least-privilege with IRSA for all service accounts
  - All secrets via External Secrets Operator + AWS Secrets Manager — no secrets in Git
  - HTTPS throughout: CloudFront → ALB → application, ACM certificate
- **AI Component**
  - At least one AI-augmented workflow: PR summaries, runbook bot, AI alert analysis, or AI-assisted IaC review
- **Documentation**
  - GitHub README: architecture diagram, tech stack table, how to deploy, how to run locally, runbook for top 3 failure scenarios
  - Two Architecture Decision Records (ADRs) explaining key design choices and their tradeoffs

🏆 PROJECT DELIVERABLES
1. GitHub repository: clean commit history, comprehensive READMEs, ADRs
2. Architecture diagram: production quality — shared on LinkedIn and in your CV
3. Blog post: detailed walkthrough of what you built, decisions made, problems encountered, what you would do differently
4. LinkedIn post with a short demo video or screenshots walking through the pipeline, the Grafana dashboard, and the running application
5. Live or recorded demo for the cohort: 10 minutes — show the pipeline running, the deployment, the monitoring

JOB-SEEKING MONTH (Advanced Track — Final 4 Weeks)

This is a structured 4-week sprint to get you hired. By the end you will have: a complete LinkedIn profile, a polished CV, a reviewed GitHub portfolio, active job applications, and interview experience. Your instructors will schedule 1:1 sessions during this month.

WEEK 1 OF JOB-SEEKING — CV, LinkedIn Finalisation & Portfolio Review

Activities

- CV / Resume
  - Format: clean, single page (max 2), reverse chronological, PDF
  - Sections: Contact → Summary → Skills → Experience → Projects → Education → Certifications
  - Skills section: AWS, Kubernetes, Docker, Terraform, GitHub Actions, ArgoCD, Prometheus, Python, Bash, Linux
  - Projects section: list your top 3 GitHub projects with a one-line description and a link — this is often more valuable than experience for entry-level roles
  - Summary section: 2–3 sentences. 'DevOps engineer with hands-on experience deploying and operating production systems on AWS using Kubernetes, Terraform, and CI/CD pipelines. Passionate about reliability and automation.'
  - Avoid: buzzword soup without evidence, generic objectives ('seeking a challenging role'), photos (in most countries)
- LinkedIn Final Audit
  - Every section filled in: About, Experience, Education, Skills (15+), Certifications (if any), Featured (links to your projects)
  - All GitHub project links added to LinkedIn Featured section
  - 'Open to Work' enabled — either visible to all or to recruiters only
  - Profile completeness: LinkedIn shows a 'Profile Strength' indicator — reach 'All-Star'
- GitHub Portfolio Review
  - Every pinned repo: clear README, architecture diagram, tech stack clearly stated
  - Profile README updated with your tech stack, current learning, and link to your blog
  - Commit history looks healthy — consistent, meaningful commits over time

✅ **Week 1 Deliverable**
Share your CV (PDF) and LinkedIn profile URL with your instructor for review.
Instructor will provide written feedback within 2 days.

WEEK 2 OF JOB-SEEKING — Job Research, Applications & Outreach

Activities

- Job Research
  - Where to look: LinkedIn Jobs, Indeed, Glassdoor, company career pages, BuiltIn (US/UK), levels.fyi (for compensation data)
  - Keywords to search: 'DevOps Engineer', 'Cloud Engineer', 'Site Reliability Engineer', 'Platform Engineer', 'Infrastructure Engineer'
  - Entry-level signals: 'junior', '0–2 years', 'associate', 'graduate', or no experience requirement stated — apply even if you don't meet every requirement
  - Target list: identify 20 companies you want to work at. Follow their LinkedIn pages. Find the hiring managers.
- Applications
  - Apply to 5–10 roles this week. Quality over quantity at first — tailor your summary and highlight relevant projects
  - After applying: find the hiring manager or recruiter on LinkedIn and send a short, genuine message 3 days later
  - Message template: 'Hi [Name], I applied for the DevOps Engineer role at [Company] on [date]. I'm particularly excited about [specific thing about the role/company]. I have hands-on experience with [relevant stack]. Would love to connect.' — Keep it short.
- Referrals & Networking

  - The highest-converting job application is a referral — ask your LinkedIn connections if anyone works at your target companies
  - Attend a meetup this week — job opportunities often come from community before they appear on job boards

✅ **Week 2 Deliverable**
Share your list of 20 target companies and 5–10 applications submitted with the cohort.
Post on LinkedIn: 'I'm actively looking for DevOps/Cloud roles. Here is what I have built [link to GitHub]. Open to opportunities.'

WEEK 3 OF JOB-SEEKING — Interview Preparation

Core Interview Topics to Prepare

- Technical Questions — Common Topics
  - Explain your CI/CD pipeline. What happens when a developer pushes code? (Walk through your final project pipeline step by step)
  - What is Kubernetes and why would you use it instead of Docker Compose?
  - A pod is in CrashLoopBackOff. Walk me through how you diagnose it.
  - Explain the AWS Shared Responsibility Model.
  - What is Infrastructure as Code? Why is it important?
  - How do you store secrets in a CI/CD pipeline?
  - What are the four golden signals of monitoring?
  - A service is returning 502 errors. How do you investigate?
- Behavioural Questions (STAR format: Situation, Task, Action, Result)
  - Tell me about a time you debugged a complex issue. What was your approach?
  - Describe a project you are proud of. What would you do differently now?
  - How do you stay current with DevOps trends and tooling?
- Prepare Your Project Story
  - Practice explaining your final project in 3 minutes without looking at notes
  - Know every component of your architecture and why you made the choices you did
  - Be honest about what you would improve — this shows maturity
- Mock Interviews
  - At least one mock interview with an instructor or a peer this week
  - Use AI: 'Act as a DevOps hiring manager. Ask me 5 technical interview questions.' Record yourself answering.

✅ **Week 3 Deliverable**
Complete one mock technical interview (recorded or with instructor).
Write answers to all 8 technical questions above and share with your instructor for feedback.

WEEK 4 OF JOB-SEEKING — Continuation, Certification & Community

Activities

- Continue Applying
  - Target 5–10 more applications. The average job seeker sends 20–50 applications before getting an offer — persistence matters
  - Follow up on previous applications that haven't responded after 10 business days
- Certifications (Optional but Valuable)
  -
  - AWS Certified Solutions Architect Associate (SAA-C03): the gold standard entry-level AWS cert — highly recognised, worth pursuing

  - CKA (Certified Kubernetes Administrator): the most respected Kubernetes cert — harder, more time investment, but strong signal for SRE and DevOps roles
  - HashiCorp Terraform Associate: validates Terraform knowledge — relatively quick to prepare for
  - Strategy: you do not need certs to get hired, but they help if you lack experience. Prioritise the SAA-C03 if AWS-focused.
- Stay Visible on LinkedIn
  - Post consistently throughout the job search — every post is a broadcast to your network that you are available and skilled
  - Share updates: 'I had my first technical interview this week. Here's what I learned.' Vulnerability and authenticity perform very well
  - Congratulate others on new roles — your generosity is remembered when people are hiring
- Build in Public
  - Continue building and learning during the job search — skills that grow during the search become talking points in interviews
  - One new small project or improvement to your portfolio per week — shows momentum

🎓 **PROGRAM COMPLETION**
By the end of this month, every graduate should have:
✓ A polished, reviewed CV and LinkedIn profile
✓ A GitHub portfolio with at least 4 strong projects (beginner capstone, month capstones, final project)
✓ A published blog post on Medium or dev.to
✓ Active job applications in progress
✓ At least one mock interview completed
✓ A professional LinkedIn network growing with DevOps practitioners
Keep going. Most offers come after 4–12 weeks of consistent searching and building. The program ends here — your career does not.

PROGRAM OVERVIEW

Week	Topic
Beginner W1	DevOps, SDLC & Engineering Team Culture
Beginner W2	Git & GitHub — Version Control Fundamentals
Beginner W3	Virtualisation: VMs, Containers & Why They Exist
Beginner W4	Linux Fundamentals & Essential Commands
Beginner W5	AWS Core Services, Principles & Architecture
Beginner W6	Shell Scripting with Bash
Beginner W7	Python Scripting for DevOps
Beginner W8	Docker: Images, Containers & Compose
Beginner W9	Git: Branching Strategies & Team Collaboration
Beginner W10	CI/CD Concepts & GitHub Actions
Beginner W11	Infrastructure as Code: CloudFormation & Terraform
Advanced W1	AI-Assisted Full-Stack Deploy & System Design
Advanced W2	Advanced Linux: Storage & Filesystems
Advanced W3	Advanced Linux: Networking & Traffic Flow
Advanced W4	Kubernetes: Architecture, Workloads & Operations
Advanced W5	Observability: Metrics, Logs, Alerts & Tracing
Advanced W6	Advanced CI/CD: Multi-Environment Pipelines & GitOps
Advanced W7	AI in DevOps: Tools, Automation & Responsible Use
Advanced W8	Major End-to-End Final Project
Job-Seeking W1	CV, LinkedIn Finalisation & Portfolio Review
Job-Seeking W2	Job Research, Applications & Outreach
Job-Seeking W3	Interview Preparation & Mock Interviews
Job-Seeking W4	Continuation, Certifications & Community